Novosibirsk State University
*THE REAL SCIENCE

Программирование на Python

Работа с файлами, обработка исключений и тестирование в Python

План лекции

1. **Работа с файлами**
   - Открытие и закрытие файлов
   - Чтение и запись
   - Контекстные менеджеры
   - Работа с путями
   - CSV и JSON
2. **Обработка исключений**
   - Концепция исключений
   - try-except-else-finally
   - Собственные исключения
3. **Тестирование**
   - Виды тестов
   - unittest и pytest
   - Best practices

Зачем нужна работа с файлами?

- Сохранение данных между запусками программы
- Чтение конфигураций
- Логирование
- Обработка больших датасетов
- Интеграция с другими системами
- Примеры: веб-скрейпинг → CSV, ML модели → сохранение весов

Базовый синтаксис открытия файлов

```
file = open('example.txt', 'r')
content = file.read()
file.close()
```

- `open()` – открывает файл
- `read()` – читает содержимое
- `close()` – закрывает файл (освобождает ресурсы)

Режимы открытия файлов

Режим	Описание
'r'	Чтение (read) – файл должен существовать
'w'	Запись (write) – перезаписывает файл
'a'	Дозапись (append) – добавляет в конец
'x'	Эксклюзивное создание – ошибка если существует
'r+'	Чтение и запись
'b'	Бинарный режим (комбинируется: 'rb', 'wb')

Контекстный менеджер with

```
# Правильный способ работы с файлами
with open('example.txt', 'r') as file:
    content = file.read()
    # работа с файлом
# файл автоматически закрыт

# Можно открыть несколько файлов
with open('input.txt', 'r') as infile, \
     open('output.txt', 'w') as outfile:
    data = infile.read()
    outfile.write(data.upper())
```

- Автоматическое закрытие
- Работает даже при исключениях
- Чистый, читаемый код

Методы чтения файлов

```
with open('example.txt', 'r') as f:
    # Прочитать весь файл
    content = f.read()

    # Прочитать одну строку
    line = f.readline()

    # Прочитать все строки в список
    lines = f.readlines()

    # Итерация по строкам (эффективно!)
    for line in f:
        print(line.strip())
```

- `read()` – небольшие файлы, нужен весь текст
- `readline()` – построчная обработка с контролем
- `readlines()` – нужен список строк
- Итерация – большие файлы (не загружает все в память)

Методы записи в файлы

```python
# Запись строки
with open('output.txt', 'w') as f:
    f.write('Hello, World!\n')
    f.write('Second line\n')

# Запись списка строк
lines = ['First\n', 'Second\n', 'Third\n']
with open('output.txt', 'w') as f:
    f.writelines(lines)

# Дозапись в существующий файл
with open('log.txt', 'a') as f:
    f.write('New log entry\n')
```

- `write()` не добавляет `\n` автоматически
- `writelines()` не добавляет разделители
- Режим 'w' удаляет существующее содержимое

Работа с путями – модуль pathlib

```python
from pathlib import Path

# Создание пути
path = Path('data') / 'input' / 'file.txt'

# Проверки
if path.exists():
    print(f"Size: {path.stat().st_size} bytes")
    print(f"Is file: {path.is_file()}")
    print(f"Parent: {path.parent}")
    print(f"Name: {path.name}")
    print(f"Extension: {path.suffix}")

# Создание директорий
output_dir = Path('output')
output_dir.mkdir(parents=True, exist_ok=True)

# Поиск файлов
for txt_file in Path('.').glob('*.txt'):
    print(txt_file)
```

Работа с CSV файлами

```
import csv

# Чтение CSV
with open('data.csv', 'r') as f:
    reader = csv.reader(f)
    headers = next(reader)
    for row in reader:
        print(row)

# Чтение CSV как словарь
with open('data.csv', 'r') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['name'], row['age'])

# Запись CSV
with open('output.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['Name', 'Age'])
    writer.writerow(['Alice', 25])
    writer.writerows([['Bob', 30], ['Charlie', 35]])
```

Работа с JSON файлами

```
import json

# Чтение JSON
with open('data.json', 'r') as f:
    data = json.load(f)

# Запись JSON
data = {
    'users': [
        {'name': 'Alice', 'age': 25},
        {'name': 'Bob', 'age': 30}
    ]
}
with open('output.json', 'w') as f:
    json.dump(data, f, indent=2, ensure_ascii=False)

# Работа со строками JSON
json_string = json.dumps(data, indent=2)
parsed_data = json.loads(json_string)
```

Кодировки при работе с файлами

```
# Явное указание кодировки (рекомендуется!)
with open('file.txt', 'r', encoding='utf-8') as f:
    content = f.read()

# Чтение файла с неизвестной кодировкой
with open('file.txt', 'r', encoding='utf-8',
          errors='ignore') as f:
    content = f.read()

# Распространенные кодировки
# utf-8 – современный стандарт
# cp1251 – Windows кириллица
# ascii – только английские символы
```

Что такое исключения?

- **Исключение (Exception)** – объект, представляющий ошибку
- Механизм обработки нештатных ситуаций
- Альтернатива возврату кодов ошибок

```
# FileNotFoundError
open('nonexistent.txt', 'r')

# ZeroDivisionError
result = 10 / 0

# ValueError
int('not a number')

# KeyError
d = {'a': 1}
print(d['b'])
```

Базовая обработка исключений – try-except

```
# Без обработки исключений
filename = input("Enter filename: ")
file = open(filename, 'r')

# С обработкой исключений
try:
    filename = input("Enter filename: ")
    file = open(filename, 'r')
    content = file.read()
    file.close()
    print(content)
except FileNotFoundError:
    print("Error: File not found!")
```

Множественные except блоки

- От конкретных к общим
- Можно обработать несколько типов сразу: `except (ValueError, TypeError):`
- `Exception` ловит все (использовать осторожно)

```
try:
    filename = input("Enter filename: ")
    with open(filename, 'r') as f:
        number = int(f.read())
        result = 100 / number
except FileNotFoundError:
    print("File not found!")
except ValueError:
    print("File doesn't contain a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:
    print(f"Unexpected error: {e}")
```

else и finally

```python
try:
    file = open('data.txt', 'r')
    data = file.read()
except FileNotFoundError:
    print("File not found")
else:
    # Выполнится, если НЕ было исключения
    print("File read successfully")
    print(f"Length: {len(data)}")
finally:
    # Выполнится всегда
    print("Cleanup complete")
```

```python
connection = None
try:
    connection = connect_to_database()
    connection.execute(query)
except DatabaseError as e:
    print(f"Database error: {e}")
finally:
    if connection:
        connection.close()  # Гарантированная очистка
```

Получение информации об исключении

```
try:
    result = int('abc')
except ValueError as e:
    print(f"Error message: {e}")
    print(f"Error type: {type(e).__name__}")

    # Полный traceback
    import traceback
    traceback.print_exc()

# Получение информации без остановки программы
try:
    dangerous_operation()
except Exception as e:
    import logging
    logging.error(f"Operation failed: {e}", exc_info=True)
    # Программа продолжает работу
```

Выброс исключений – raise

```python
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive")
    if amount > balance:
        raise ValueError("Insufficient funds")
    return balance - amount

# Использование
try:
    new_balance = withdraw(100, 150)
except ValueError as e:
    print(f"Transaction failed: {e}")

# Повторный выброс исключения
try:
    process_file()
except PermissionError:
    print("Logging error...")
    raise  # Пробрасываем исключение дальше
```

Создание собственных исключений

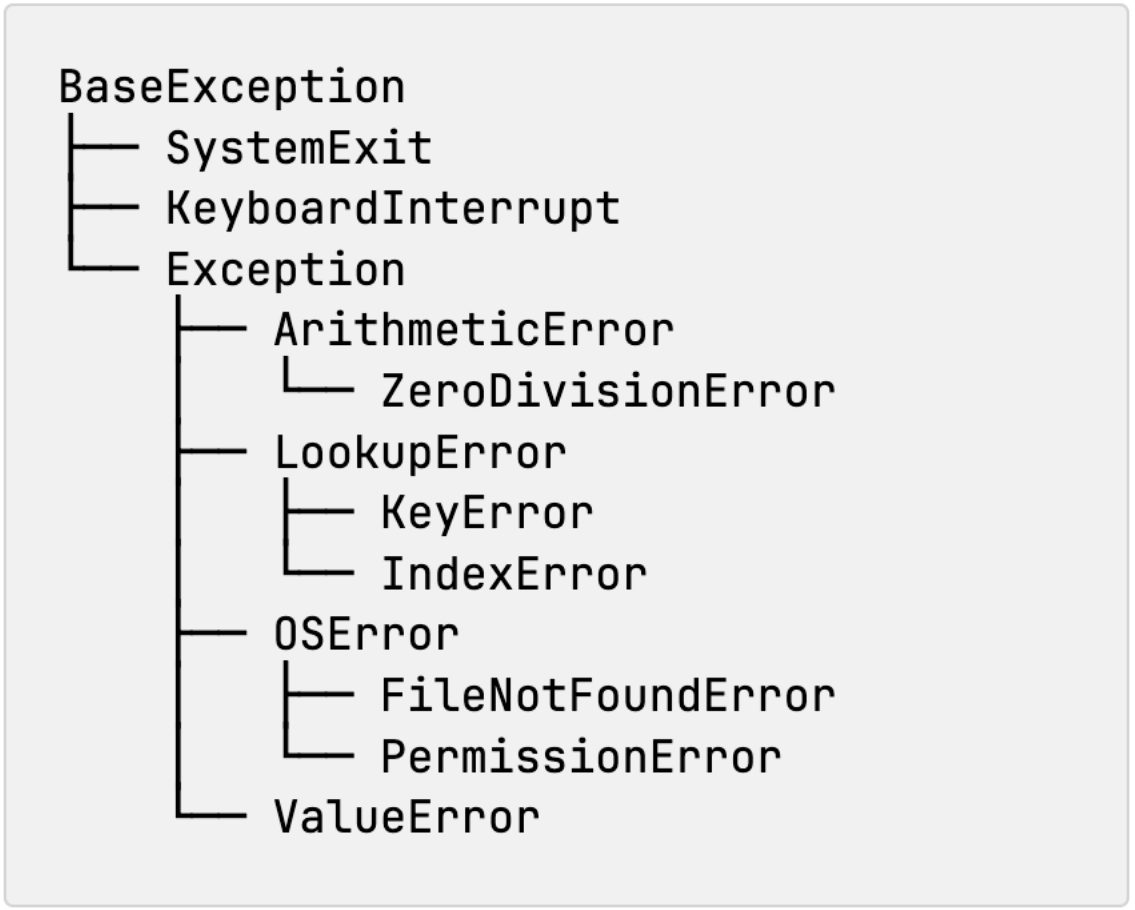
```
# Собственное исключение
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(
            f"Cannot withdraw {amount}. Balance: {balance}"
        )

class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFundsError(
                self.balance, amount
            )
        self.balance -= amount

# Использование
try:
    account = BankAccount(100)
    account.withdraw(150)
except InsufficientFundsError as e:
    print(f"Error: {e}")
    print(f"Available: {e.balance}")
```

Иерархия исключений и best practices



- Не ловите все except
- Ловите конкретные исключения
- Не используйте исключения для контроля потока
- Используйте для исключительных ситуаций
- Fail fast – падайте рано и понятно
- Логируйте исключения в production

Зачем нужно тестирование?

- **Уверенность в коде** – изменения не ломают функциональность
- **Документация** – тесты показывают, как использовать код
- **Упрощение рефакторинга** – можно менять реализацию безопасно
- **Раннее обнаружение багов** – дешевле исправить на этапе разработки
- **Лучший дизайн** – тестируемый код = хорошо спроектированный код

Виды тестов

1. **Unit-тесты (модульные)**
   - Тестируют отдельные функции/методы
   - Быстрые, изолированные
   - Основа тестовой пирамиды
2. **Integration-тесты (интеграционные)**
   - Тестируют взаимодействие компонентов
   - Работа с БД, API
3. **End-to-End (E2E)**
   - Тестируют систему целиком
   - Имитируют действия пользователя
   - Медленные, но дают максимальную уверенность

Модуль unittest – стандартная библиотека

```
import unittest

def add(a, b):
    return a + b

class TestMath(unittest.TestCase):
    def test_add_positive(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative(self):
        self.assertEqual(add(-1, -1), -2)

    def test_add_zero(self):
        self.assertEqual(add(5, 0), 5)

if __name__ == '__main__':
    unittest.main()
```

Основные методы assert в unittest

```
class TestAssertions(unittest.TestCase):
    def test_equality(self):
        self.assertEqual(a, b)
        self.assertNotEqual(a, b)

    def test_booleans(self):
        self.assertTrue(x)
        self.assertFalse(x)
        self.assertIsNone(x)
        self.assertIsNotNone(x)

    def test_membership(self):
        self.assertIn(item, container)
        self.assertNotIn(item, container)

    def test_exceptions(self):
        with self.assertRaises(ValueError):
            int('not a number')

    def test_floats(self):
        self.assertAlmostEqual(0.1 + 0.2, 0.3, places=7)
```

setUp и tearDown

```
class TestFileOperations(unittest.TestCase):
    def setUp(self):
        # Выполняется перед каждым тестом
        self.test_file = 'test_data.txt'
        with open(self.test_file, 'w') as f:
            f.write('test content')

    def tearDown(self):
        # Выполняется после каждого теста
        if os.path.exists(self.test_file):
            os.remove(self.test_file)

    def test_read_file(self):
        with open(self.test_file, 'r') as f:
            content = f.read()
        self.assertEqual(content, 'test content')

    def test_append_file(self):
        with open(self.test_file, 'a') as f:
            f.write(' more')
        # Файл будет удален в tearDown
```

pytest – современная альтернатива

```python
def add(a, b):
    return a + b

def test_add_positive():
    assert add(2, 3) == 5

def test_add_negative():
    assert add(-1, -1) == -2

def test_add_raises():
    import pytest
    with pytest.raises(TypeError):
        add('a', 'b')  # если add ожидает числа
```

- Проще синтаксис (обычные функции)
- Лучший вывод при ошибках
- Мощные fixtures
- Автоматическое обнаружение тестов

Fixtures в pytest

```python
import pytest

@pytest.fixture
def sample_file(tmp_path):
    file = tmp_path / "test.txt"
    file.write_text("test content")
    return file

def test_read_file(sample_file):
    content = sample_file.read_text()
    assert content == "test content"

def test_file_exists(sample_file):
    assert sample_file.exists()

# Fixture с cleanup
@pytest.fixture
def database_connection():
    conn = connect_to_db()
    yield conn  # тест выполняется здесь
    conn.close()  # cleanup после теста
```

Параметризация тестов

```python
import pytest

# unittest подход - много похожих тестов
def test_add_case1(self):
    self.assertEqual(add(2, 3), 5)
def test_add_case2(self):
    self.assertEqual(add(-1, 1), 0)
def test_add_case3(self):
    self.assertEqual(add(0, 0), 0)

# pytest подход - параметризация
@pytest.mark.parametrize("a, b, expected", [
    (2, 3, 5),
    (-1, 1, 0),
    (0, 0, 0),
    (100, -50, 50),
])
def test_add(a, b, expected):
    assert add(a, b) == expected
```

Что тестировать? Best practices

- Публичный API функций и классов
- Граничные случаи (0, -1, пустые коллекции)
- Обработку ошибок и исключений
- Сложную бизнес-логику

- Тривиальные геттеры/сеттеры
- Приватные методы напрямую
- Код сторонних библиотек

Что тестировать? Best practices

- **Arrange-Act-Assert** (Подготовка-Действие-Проверка)
- **Один тест = одна проверка**
- **Тесты должны быть быстрыми**
- **Тесты должны быть независимыми**
- **Тесты должны быть детерминированными**

Тестовое покрытие (Coverage)

- **Интерпретация:**
  - Coverage показывает, какой % кода выполнился в тестах
  - 80%+ считается хорошим покрытием
  - 100% покрытие ≠ отсутствие багов
  - Используйте как ориентир, не как цель
- **Что показывает:**
  - Какие строки не покрыты тестами
  - Какие ветки if/else не проверены

```
# Установка
pip install pytest-cov

# Запуск с измерением покрытия
pytest --cov=mymodule tests/

# Генерация HTML отчета
pytest --cov=mymodule --cov-report=html tests/
```

TDD (Test-Driven Development)

- **Red** – напишите падающий тест
- **Green** – напишите минимальный код для прохождения теста
- **Refactor** – улучшите код, тесты остаются зелеными

```python
# 1. Red - пишем тест
def test_calculate_discount():
    assert calculate_discount(100, 10) == 90

# 2. Green - минимальная реализация
def calculate_discount(price, discount_percent):
    return price - (price * discount_percent / 100)

# 3. Refactor - улучшаем
def calculate_discount(price, discount_percent):
    if not 0 <= discount_percent <= 100:
        raise ValueError("Discount must be 0-100")
    return price * (1 - discount_percent / 100)
```

Интеграция всех тем

```
# Функция с обработкой исключений
def save_user_data(filename, user_data):
    """Save user data to JSON file."""
    try:
        with open(filename, 'w', encoding='utf-8') as f:
            json.dump(user_data, f, indent=2)
    except PermissionError:
        raise PermissionError(f"No permission to write {filename}")
    except Exception as e:
        raise IOError(f"Failed to save data: {e}")

# Тест
def test_save_user_data(tmp_path):
    file = tmp_path / "user.json"
    user = {"name": "Alice", "age": 25}

    save_user_data(str(file), user)

    assert file.exists()
    with open(file, 'r', encoding='utf-8') as f:
        loaded = json.load(f)
    assert loaded == user
```

Полезные материалы

- https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files
- https://docs.python.org/3/tutorial/errors.html
- https://docs.python.org/3/library/pathlib.html
- https://docs.python.org/3/library/csv.html
- https://docs.python.org/3/library/json.html
- https://docs.pytest.org/
- https://docs.python.org/3/library/unittest.html
- https://pytest-cov.readthedocs.io/